Research article

An open architecture for sensory feedback control of a dual-arm industrial robotic cell

Vincenzo Lippiello, Luigi Villani and Bruno Siciliano
PRISMA Lab – Dipartimento di Informatica e Sistemistica, Università di Napoli Federico II, Napoli, Italy

Abstract
Purpose – To present an open architecture for real-time sensory feedback control of a dual-arm industrial robotic cell. The setup is composed of two industrial robot manipulators equipped with force/torque sensors and pneumatic grippers, a vision system and a belt conveyor.
Design/methodology/approach – The original industrial robot controllers have been replaced by a single PC with software running under a real-time variant of the Linux operative system.
Findings – The new control architecture allows advanced control schemes to be developed and tested for the single robots and for the dual-arm robotic cell, including force control and visual servoing tasks.
Originality/value – An advanced user interface and a simulation environment have been developed, which permit fast, safe and reliable prototyping of planning and control algorithms.

Keywords Real-time scheduling, Robotics, Rapid prototypes

Paper type Research paper

1. Introduction

The development of advanced control algorithms for industrial robots requires open control architectures in which software modules can be modified and external hardware such as force/torque sensors and vision systems can easily be integrated. Rapid prototyping (i.e. the ability to design and test new control and supervision algorithms in a short time and at limited cost) is becoming a fundamental issue in industrial robotic applications. Hence, the availability of flexible and highly reconfigurable robotic units is of the utmost importance.

Various open control architectures for industrial robots have already been developed by robot and control manufacturers as well as in research laboratories (www.mitsubishielectric.com, www.robot.lth.se). However, the "degree of openness" in a robot controller may vary from one system to another. Usually some components, for example, the power system and the low level control, are proprietary and cannot be modified by the user, whilst others such as the communication interface hardware and the higher level control may be considered open (i.e. based on standard hardware and software with open interface specifications).

Most of the existing open robot architectures are based on standard PC hardware with a standard operating system. In fact, a PC-based controller can more easily integrate many commercially available add-on peripherals such as mass storage devices, Ethernet card and other input/output devices. Moreover, standard software development tools such as Visual C++ Visual Basic and Delphi, can be utilized.

An important issue of control software architectures concerns real-time operating systems. In recent years, the real-time variants of the Linux operating system have been widely adopted, especially in research laboratories (www.gnu.org, Macchelli and Melchiorri, 2002). This choice is motivated by the fact that Linux and its two main real-time variants RT-Linux (www.rtlinux.org) and RTAI-Linux (www.rtai.org) are distributed under the GNU public licence (www.orocos.org), so they are freely available and configurable to meet desired requirements. Moreover, all the source code, as well as powerful development tools and detailed documentation, are accessible.

In this paper, we present an environment for open real-time control of the industrial cell of the PRISMA Lab (www.prisma.unina.it), based on two Comau SMART-3 S robots. The control architecture uses the open version of the industrial Comau C3G 9000 controller (Dogliani *et al.*, 1993), produced by Tecnospazio SpA, which allows the robot to be controlled from a standard PC working with the MS-DOS operating system. In the new open controller, named RePLiCS, the software on the PC was completely replaced by a real-time control environment based on the RTAI-Linux operating system (Caccavale *et al.*, 2005).

The current issue and full text archive of this journal is available at
www.emeraldinsight.com/0143-991X.htm

Industrial Robot: An International Journal
34/1 (2007) 46–53
 [ISSN 0143-991X]
[DOI 10.1108/01439910710718441]

The authors wish to thank Nello Grimaldi for significant support in the development of RePLiCS. The contribution of Fabrizio Caccavale on the dual-arm robotic cell is also gratefully acknowledged.

Industrial Robot: An International Journal
Volume 34 · Number 1 · 2007 · 46–53

RePLiCS allows advanced control schemes to be designed and tested, including force control and visual servo control. An advanced user interface and a simulation environment have also been developed, permitting the fast, safe and reliable prototyping of planning and control algorithms.

A notable feature of RePLiCS, which is an enhancement of existing industrial multi-robot controllers, is that it allows not just only the time synchronization of the sequence of operations executed by each robot, but also real cooperation between the robots. Two kinds of cooperation are possible: loose cooperation and tight cooperation. Loose cooperation means that the robots do not physically interact during the task execution, for example, one robot moves a work-piece while the other performs a laser welding process on it. Tight cooperation means that the robots physically interact through a common manipulated object, for example, two or more robots cooperate to transport the same heavy or large object.

Moreover, an enhanced vision system named VESPRO has been developed and interfaced with RePLiCS, and it is able to manage a multi-camera visual system (Lippiello *et al.*, 2005). This system is designed to perform visual pose (position and orientation) estimation of a moving object with known geometry.

2. Potential real-world applications

The control architecture presented in this paper is not yet ready for use in commercial industrial robots. However, it is a valuable tool for developing and testing advanced control schemes to be implemented in standard industrial control architectures. Multi-robot coordination, force control and visual servo-controlling are areas of ongoing major industrial robot control developments.

Various approaches to multi-robot control have already been presented by several robot manufacturers, and some products are already available on the market, such as the multi-move function offered by ABB (www.abb.com/robotics). Multi-robot control in industry leads to a reduction of production costs by having robots work in parallel, especially in low speed processes such as arc welding. Multiple robots can be used to carry large and/or heavy loads. Other benefits are that several robots can be controlled from one controller, floor space is saved, collision avoidance is improved and cycle times reduced.

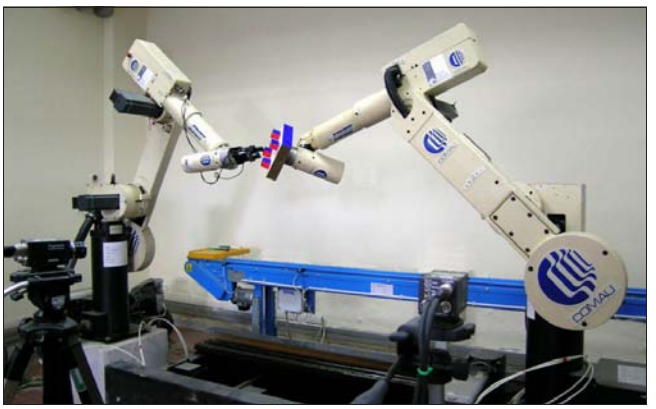
Examples of applications where 6-DOF force/torque sensors can control industrial robots are grinding, deflashing, deburring, milling, polishing, testing and assembly. In the material removal applications, the advantages of force-sensor based control are higher process quality, easier calibration and reduced requirements on the accuracy of fixtures and grippers (Pires *et al.*, 2002). Additional advantages in assembly are shorter cycle times, reduced impact forces and a lower risk of jamming, wedging and galling (Zhang *et al.*, 2004).

Vision can increase the flexibility in material handling, machine tending and assembly (www.memagazine.org/contents/current/features/eyeson/eyeson.html). Moreover, 3D vision techniques can be adopted for the calibration of tools, work objects, fixtures and other robot cell components.

3. The experimental setup

The setup in the PRISMA Lab consists of two Comau SMART-3 S industrial robots (Figure 1). Each robot manipulator has a six-axis anthropomorphic geometry with non-null shoulder and elbow offsets and non-spherical wrist. One manipulator is mounted on a sliding track that provides an additional degree of mobility. The joints are actuated by brushless motors via gear trains; shaft absolute resolvers provide motor position measurements.

Figure 1 The dual-arm industrial robotic cell



Each robot is controlled by the C3G 9000 control unit, which has a VME-based architecture with two processing boards (Servo CPU and Robot CPU) both based on a Motorola 68020/68882. The Servo CPU has an additional digital signal processor and manages trajectory generation, direct and inverse kinematics, micro-interpolation of the joint references and joint position servo control; independent joint control is adopted, in which the individual servos are implemented as standard PID controllers. The Robot CPU handles the man-machine interface and interprets the user's programs written in the PDL 2 programming language. This board also includes a shared memory area accessible by the other boards connected to the VME bus.

Upon request, COMAU supplies the proprietary controller unit with a BIT3 bus adapter board, to connect the VME bus of the C3G 9000 unit to the ISA bus of a standard PC with an MS-DOS operating system, so that the PC and C3G controller communicate via shared memory in the Robot CPU. In this way the PC can implement control algorithms (Dogliani *et al.*, 1993), and time synchronization is achieved using a flag set by the C3G and read by the PC. A closed proprietary C library (PCC3Link produced by Technospazio SpA) is available to perform communication tasks, for example, reading shaft motor positions and/or writing motor reference currents or reference positions from/to the shared memory.

A schematic of the open control architecture for one robot is shown in Figure 2.

Seven different operating modes are available in the C3G control unit, allowing the PC to interact with the original controller both at trajectory-generation level and at joint-control level. The most useful operating modes are numbers 4 and 6, which are conceptually shown in Figure 3.

In Operating Mode 4, the joint position servos managed by the C3G are opened, and the PC is responsible for acquiring data from the resolvers, computing the control algorithm and passing the references to the current servos at 1 ms sampling time. Hence, the C3G controller is used as an interface between the PC and the resolvers and the brushless motors of the robot.

Figure 2 Schematic of the C3G open control architecture for one robot

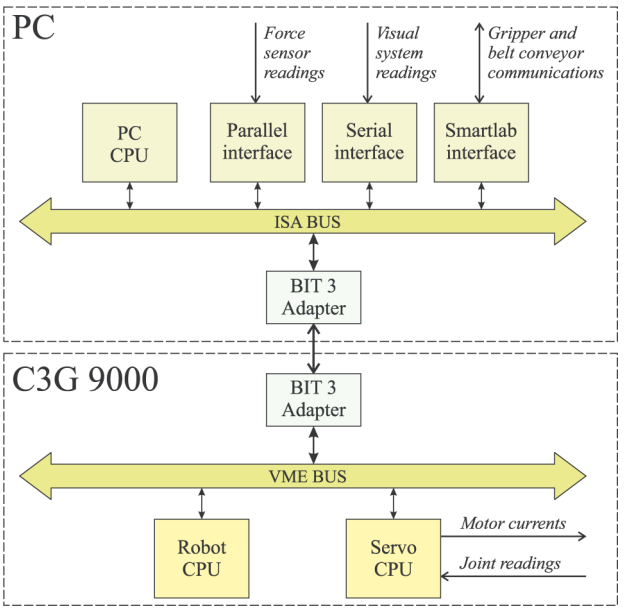
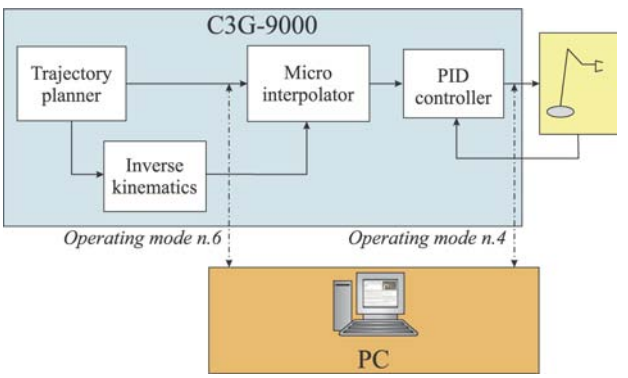


Figure 3 The two operating modes of the C3G open controller



In Operating Mode 6, the PC computes the joint position references for the micro-interpolator of the Servo CPU of the C3G controller at 2 ms sampling time. Thus, the PC handles the trajectory planning and kinematic inversion, whilst the native joint position servos of the C3G controller move the robot.

The sensing capabilities of the robotic cell are completed by force/torque sensors and a stereo vision system. An ATI FT30-100 six-axis force/torque sensor with a force range of ± 130 N and a torque range of ± 10 Nm can be mounted at the wrist of either arm. The sensor is connected to the PC by a parallel interface board which provides readings of six components of generalized force at 1 ms intervals. The vision system is composed of a PC with a Pentium IV 1.7 GHz processor and Windows XP operating system, equipped with two Matrox Genesis boards and two Sony 8500CE B/W cameras. The Matrox boards are used as frame grabbers as well as partial image processors (e.g. for window extraction from the image), while the PC host is responsible for executing vision-based algorithms and communicating with the PC performing robot control, via a standard serial and/or parallel connection.

Each robot can also be equipped with a pneumatic gripper with two parallel jaws. The completion of the open and close operations are detected by Hall-effect sensors. The grippers can be directly controlled by the C3G industrial controller using PDL2 instructions, or by the PC through a SMARTLAB ISA interface board. Through this board it is also possible to operate a belt conveyor installed at one side of the cell, parallel to the two robots, which can be driven in the two directions and which has an optical presence sensor at the two sides and a magnetic sensor for the identification of particular work-pieces carried by the belt.

Figure 3 shows the complete control architecture for one robot, including force/torque sensor, vision, gripper and belt conveyor. In the case of the dual-arm robotic system, a single PC controls the whole cell. Hence, on the ISA bus of the PC there are two different BIT3 boards, which allow connection with the two separate VME buses of the C3G 9000 controllers, two parallel interfaces for the two force/torque sensors, one serial interface for the vision system and one SMARTLAB interface for the two grippers and the belt conveyor.

The above experimental setup, due to the open control architecture, allows the design and testing of advanced control algorithms, both for the single robot and for the two cooperating robots. A large number of experimental tests have been carried out during recent years, including kinematic control (Caccavale *et al.*, 1997), resolved acceleration control (Caccavale *et al.*, 1998), force/position control (Chiaverini *et al.*, 1999), dual-arm manipulator control (Caccavale *et al.*, 2001), visual servo control and tracking (Lippiello and Villani, 2003).

However, the system suffers many problems due, essentially, to the limitations of the MS-DOS operating system. The main drawbacks are:

- the RAM available for programs and data storage is limited to 512 kb;
- the integration of advanced sensing capabilities, such as vision, is not easy;
- the coordination between the two robots is difficult because a tight time synchronization between the two control units cannot be realized;
- the system is not multitasking and it is not possible to build a user interface to display data or input commands while the controller is running; and
- the robotic cell cannot be connected to the internet.

In view of these limitations, a new real-time control system has been designed for the cooperative cell available at the PRISMA Lab. The aim was to create a flexible experimental set-up that allows the robots to be used in different tasks. These may involve a single robot, time synchronization of the two robots, grippers and conveyor belt, as well as loose and tight cooperation of the dual-arm system. Further, requirements were the possibility of executing interactive tasks or tight cooperation tasks using force control and visual servo-control. Last but not least, a crucial requirement was the development of an advanced user interface for monitoring the system during the experiment as well as for rapid prototyping of task planning and control algorithms.

4. RePLiCS

The new control software, named RePLiCS (REal-time PrismaLab LInux Control System), was developed using RTAI-Linux, which is a real-time variant of the Linux operating system.

RePLiCS is structured as a real-time module (that is a driver for the kernel of RTAI-Linux), plus a set of non real-time applications to provide a user interface for the real-time module. The real-time module is periodically activated by an external interrupt signal generated by the C3G 9000 controller. There are appropriate communication channels between the real-time module and the user applications.

4.1 RePLiCS real-time module

The real-time module of RePLiCS implements all the real-time functions required for the control of the robotic cell. All these functions are collected in an API (Application Programming Interface) software library written in the C language. These functions are grouped as follows:

- communication with the C3G-9000 controllers;
- reading the force sensors;
- synchronization for cooperative control;
- safety checks;
- robot kinematics;
- robot control;
- trajectory planning;
- serial and parallel communication;
- data storage; and
- I/O functions (file or console).

The functions giving communication with the C3G-9000 controllers implement the drive on/off commands and access the shared memory area of the Robot CPUs. They read the joint positions and write the current set–points or the position set-points, depending on the selected operating mode. For example, in Operating Mode 4, the C3G executes the following operations:

- set the synchronization flag IntActive (interrupt signal);
- write the values of the motor shaft angular positions; and
- read the desired values for the motor current set-points.

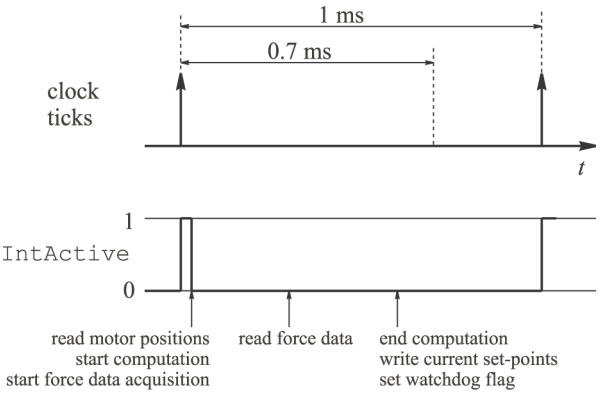
Meanwhile, the PC implements the following operations:

- reset the synchronization flag IntActive;
- read the values of the motor shaft angular positions;
- compute the current set-points for the next time interval;
- write the desired values for the motor current set-points; and
- set the watchdog flag.

To fulfill the real-time constraint at 1 ms sampling time, the PC must execute the above steps within 0.7 ms. In the remaining 0.3 ms time window, called the "forbidden window" the PC must not attempt to access the shared memory. The sequence outlined above is shown in Figure 4. The watchdog flag implements a watchdog timer mechanism, which guards against harmful consequences of unpredictable events, for example, when the PC crashes and is no longer able to write a correct new current set-point. If this situation occurs, the system enters an alarm state, the robot drives are switched off and brakes are activated.

To obtain the force measurements, the PC sends a data acquisition request, which starts the data A/D conversion on the sensor-conditioning electronics. After a time lapse of about 0.25 ms, the six components of the force and torque are

Figure 4 Timing diagram of the real-time control loop



available in a memory buffer. Since, the conversion is executed on remote hardware, the PC can continue its processing during the conversion time interval. To avoid simply waiting for the end of conversion, it is convenient start the conversion at the beginning of the control cycle, perform all the computations not requiring force measurements, e.g. kinematics, dynamic model compensation, then read the force data and, finally, complete the control algorithm. The timing of these operations is shown in Figure 4.

To implement the cooperative control of the two robots, the PC should write the positions or the current set-points for both robots at the same time, using motor angular positions read at the same time. This ideal behavior, however, is difficult to achieve because the two C3G controllers have separate clocks. Hence, the temporal windows for reading and writing on the shared memory, as well as the forbidden windows, are usually not aligned. Moreover, since the two clock frequencies are slightly different, a time drift occurs between the two trains of clock ticks. To solve this problem the synchronization module of RePLiCS, in a preliminary phase, checks the state of the IntActive flags and the watchdog flags of the two C3G controllers and defines one robot as leader and the other as follower. The follower is the robot whose clock tick does not arrive during the forbidden window of the other robot (see Figure 5, which shows the forbidden windows of the two clock trains in red).

At this point, the control loop can start, and the following operations are executed:

- write the desired values for the motor current set-points of both robots;
- read the values of the motor shaft angular positions of both robots;
- set the watchdog flags; and
- compute the current set-points for the next time interval.

The time shift between the two clocks may cause the situation shown in Figure 6, in which the clock tick of the follower

Figure 5 Timing diagram of the leader and follower clocks

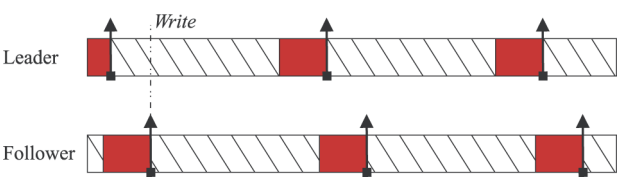
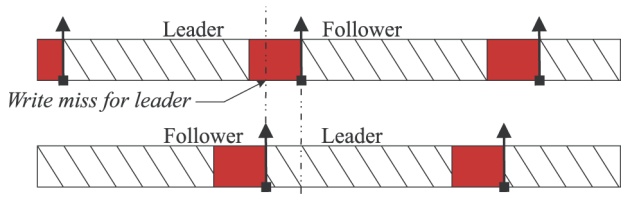


Figure 6 Exchange of role between leader and follower



arrives during the forbidden window of the leader. In this case, the role of the leader and of the follower are exchanged, so that the computed current set-points are written when the clock tick of the new leader arrives. This implies that the previous leader loses the reference only for one time interval, which is insignificant. Particular solutions must be adopted for handling singular cases, for example, when the clocks of the leader and the follower are almost aligned.

In addition to the watchdog timer, several safety checks can be carried out to monitor the system and prevent damage. This includes setting and checking joint limits, maximum joint velocities, maximum instantaneous currents, maximum sustained currents, and maximum forces and torques.

Two different approaches can be taken when one of these checks fails: to stop the robot immediately by invoking a special emergency routine, or to set an error flag to exit from the main control loop and switch off the drives. The former approach is faster and safer, but requires a complete reboot of the system. The latter approach gives a delay of one sampling period, but leads the system to a less critical state that does not require a complete reboot. The best policy is to base the decision on the particular safety check that failed: for example, if one of the motor currents exceeded its maximum sustained value, an immediate stop is not necessary; on the other hand, if the force is over the maximum allowed value, an immediate stop is advisable.

The robot kinematics functions allow the computation of the direct kinematics and their Jacobians. The inverse kinematics are computed by means of CLIK algorithms (Caccavale *et al.*, 2001). The damped least-squares inverse Jacobian is adopted to cope with singularity problems. Several utility functions are available, e.g. for converting the units of the joint variables (degrees, radians, and bit resolvers), for converting the coordinates between different frames, for representing the orientation (Euler angles, angle/axis, quaternion). The kinematic library is still under development and will include modules for redundancy resolution, and inverse kinematics for dual-arm systems.

The robot control functions implement decentralized joint control as well as centralized control, and include inverse dynamics and resolved acceleration in the task space (Caccavale *et al.*, 1998). Interaction control strategies (Chiaverini *et al.*, 1999) are also implemented, based on force measurements. Moreover, there are software modules that realize loose and tight cooperation (Caccavale and Villani, 2000). New control schemes can easily be programmed by modifying the control module of a template program file (written using the standard C language), and this includes the API library of RePLiCS.

To implement trajectory planning, there is a set of functions for point-to-point motion, in both joint and task space (along straight lines for the position). These use time laws with a trapezoidal velocity profile. There is also a function to generate a path in the joint space or task space via specified points. Special functions ensure synchronization of the two robots at the trajectory planning level, and generate smooth trajectories when the target is not known in advance, for example, in visual servo-control applications).

Serial and parallel communications allow the controller to communicate with an external device (e.g. the PC used for the vision system) by means of standard serial and parallel ports. A set of functions have been developed to control the two grippers and the belt conveyor through the SMARTLAB interface board.

Special functions of the API library manage the storage of significant variables to record their time history in a given experiment, because of the real-time constraints, these values are saved in the RAM memory of the PC along with an assigned sample time, and they are saved on the hard disk only at the end of the experiment. This procedure can be guided by facilities within user applications software.

Finally, the real-time module of RePLiCS includes I/0 functions for files and console. These functions cannot be executed while the robot control is active, because they may cause a watchdog alarm. A special monitor application within RePLiCS suspends these instructions and allows their execution only in the absence of real-time constraints.

All the real-time functions are compiled in a kernel module and dynamically linked to the real-time kernel of the operating system. Since, the user needs to interact with the robot, there are communication channels between the kernel space and the user space.

From the user space it is possible, for example, to send the drive on/off command, or change the joint limits and the other safety checks, or open or close the grippers. The real-time RePLiCS module can receive information about the desired joint or end-effector set-points, and return the current values of the robot's internal variables (positions, velocities, motor currents and, contact force). In this way, it is possible to move the robots by a virtual teach–pendant or to display the internal variables on the screen while the robots are moving.

4.2 RePLiCS user applications

The software applications in the user space essentially assist the human user to communicate with the dual-arm robotic cell through a Graphical User Interface (GUI).

In Figure 7, all the GUI windows are collected in the same graphical page.

The window on the top-left of Figure 7, which is also shown in Figure 8, is the main RePLiCS GUI which facilitates the most important operations on the system. In particular, using the menu bar or toolbar, it is possible: to select one or both robots, select the operating mode (4 or 6), send drive on/off commands, select the type of motion (joint space or task space) and select the real or virtual mode of operation for RePLiCS. This latter feature is of prime importance because it enables the testing of all the functionalities of the controller in a simulation environment that respects the real-time constraint and includes the C3G 9000 controllers, the robot dynamics and the interaction with a virtual environment. Hence, the whole control prototyping process can be developed off line in a very fast, safe and reliable way. Moreover, the code developed in virtual mode can be executed in the real mode with no modification, using the real measurements from any external sensors.

Figure 7 A screen-shot of all the RePLiCS GUI

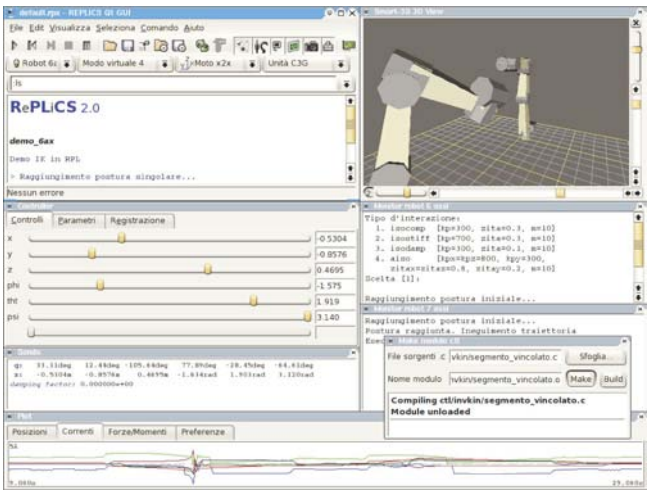


Figure 8 A screen-shot of the GUI of the main user application

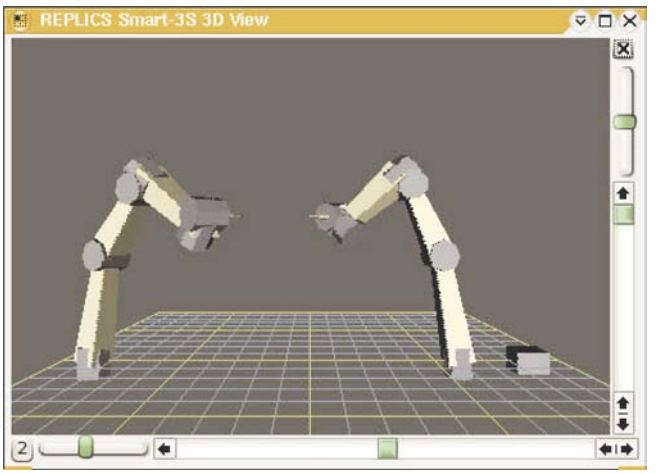
From the main window, it is also possible to set the joint limits, the maximum currents and other safety checks, select a point-to-point motion in the joint space or in the task space, set the parameters of the velocity-profile time law, start the selected motion, pause the motion before completion and resume it. For task space motions, the GUI allows parameter selection for the CLIK algorithm, for computation of the corresponding joint motions. All parameters are set in the Controller window on the middle-left of Figure 7. This window also implements a virtual teach pendant to move the robots in joint space or task space, and enables the selection of variables for recording in the RAM memory of the PC. These variables are displayed in the Plot window (the window on the bottom of Figure 7) during task execution and can be stored on the hard disk at the end of the operation.

A 3D graphical representation of the dual robot system allows the user to dynamically change the point of view and zoom setting and the graphical window is continuously updated during task execution (Figure 9).

From the main window it is also possible to compile and execute the user-written control modules (linked to the real-time kernel). The console input/output of the control modules is realized through the Monitor window on the middle-right of Figure 8.

To facilitate the interaction between the user and the robotic cell, a simple programming language named RPL

Figure 9 3D representation of the dual-arm system



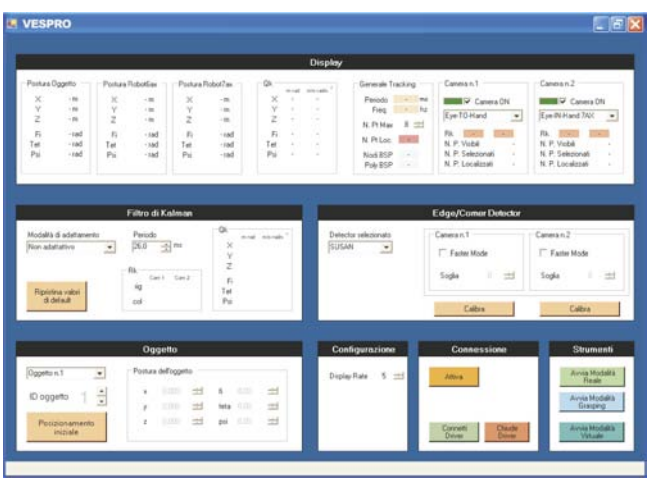
(RePLiCS Programming Language) has been developed, and an RPL interpreter has been produced. The RPL instructions are input through the console of the RePLiCS main window (Figure 8), or grouped in script files that are executed as batch programs. These instructions also program synchronized tasks for the two arms of the cell, the grippers and the belt conveyor. Further, details on the RPL language and on RePLiCS software can be found in Grimaldi (2004).

4.3 VESPRO

VESPRO (Visual Enhanced System for PrismaLab Robot Operation) is a software environment that manages a multi-camera visual system for pose estimation of moving objects with known geometry. It is structured as a low-level driver written entirely in the C and in C++ languages, with a GUI for the Windows NT operating system (Figure 10).

This visual system may be used to perform both position-based and image-based visual servo control tests. For brevity, only position-based visual servoing is considered in this paper. The vision system estimates the pose of a target object with respect to a reference frame, and the estimate is then passed to a pose controller. The two main operations are pose control and pose estimation.

Figure 10 A screen-shot of the GUI of VESPRO



Pose estimation is a computationally demanding task that involves processing the measurements of geometric features extracted from the images from one or more cameras. So the sampling time of the pose estimation algorithm is usually higher than that of the pose control loop. In the best case, the pose estimation can be performed at camera frame rate (between 25 and 60 Hz).

Figure 11 shows a schematic diagram of the position-based visual servoing algorithm implemented in the experimental setup.

The pose control is performed using an inner-outer control loop. The inner loop, running at 2 ms sampling time, implements motion control (independent joint control or any kind of joint space or task space control). The block named "dynamic trajectory planner" computes the specific trajectory for the end-effector for the desired task on the basis of the current object pose. The output of this block is available at 500 Hz frequency, while the input is updated only at 26 Hz, corresponding to the frame rate of the cameras. RePLiCS implements the pose control whilst VESPRO implements the pose estimation algorithm under Windows NT on a dedicated PC, and provides the measurements of the target object pose at a frequency of 26 Hz.

The vision system may be based on eye-in-hand cameras, i.e. one or two cameras mounted on the robot end-effector, or it can be based on a system of multiple fixed cameras. Hybrid configurations, including both eye-in-hand and fixed cameras may be a good solution, integrating the advantages of both configurations.

The use of a multiple camera system requires intelligent and efficient strategies for managing highly redundant information (a large number of image object features from multiple points of view). This is realized with hard time constraints and thus it is not possible to extract and interpret all the available visual information. To solve this problem, an efficient technique has been developed to improve the accuracy and robustness of the visual system by exploiting a minimal set of significant information selected from the initial redundant set. This technique, implemented in VESPRO, is described in Lippiello and Villani (2003).

The software architecture of VESPRO is shown in Figure 12. The driver comprises a collection of software modules activated and configured by a Module Manager that translates the high level commands from the GUI.

The hardware interface module controls the interfacing with the two MATROX Genesis boards for frame grabbing and image pre-processing. In particular, the low-level functions that control the boards are integrated into high-level functions

Figure 11 Position based visual servoing scheme

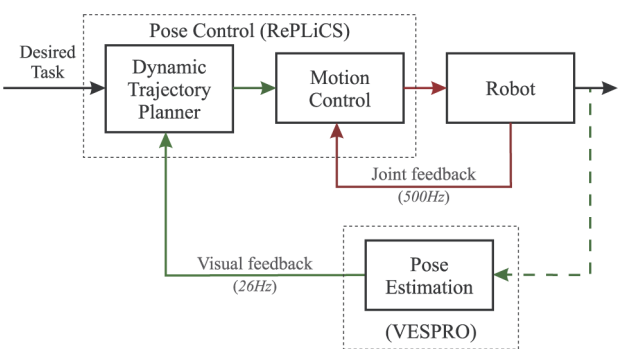
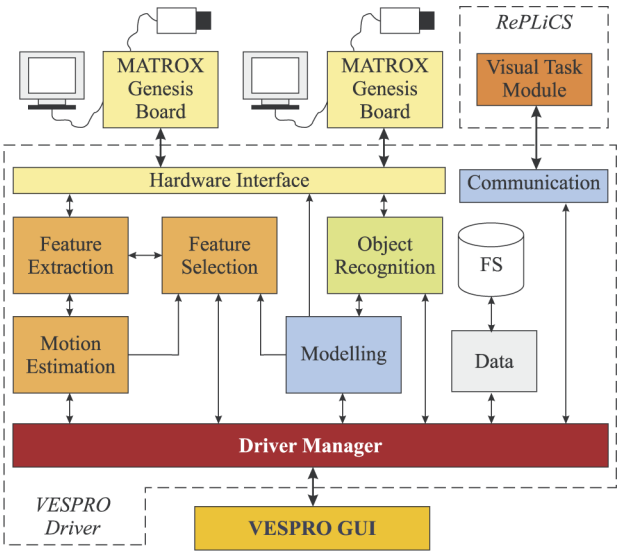


Figure 12 Schematic of the VESPRO software architecture



that allow, for example, image acquisition, image window extraction, and image preprocessing such as binarization and convolution. This forms a unique VESPRO module that is hardware dependent (i.e. specific to the particular frame grabber boards). Moreover, it includes a software library for characteriing the cameras and compensating for geometrical distortion. Note that VESPRO manages both fixed cameras and eye-in-hand cameras.

The modeling module manages a library of CAD models of objects and builds BSP-tree representations of a single objects or a set of objects.

5. Conclusion

In this work, an open architecture for real-time sensory feedback control of an industrial robotic cell based on RTAI-Linux has been described. The environment allows fast prototyping of advanced control algorithms, both for a single robot and for two cooperating robots. Force/torque sensors and vision sensors can be included. Future work further improve the features of the current system, for example, build a library of advanced control modules, enrich the RPL language of more powerful instructions, develop internet tools for programming and operating the robotic cell from a remote location.

References

Caccavale, F. and Villani, L. (2000), "Impedance control of cooperative manipulators", *Machine, Intelligence & Robotic Control*, Vol. 2, pp. 51-7.

Caccavale, F., Chiaverini, S. and Siciliano, B. (1997), "Second-order kinematic control of robot manipulators with Jacobian damped least-squares in-verse: theory and experiments", *IEEE/ASME Transactions on Mechatronics*, Vol. 2, pp. 188-94.

Caccavale, F., Lippiello, V., Siciliano, B. and Villani, L. (2005), "RePLiCS: an environment for open real-time control of a dual-arm industrial robotic cell based on

RTAI-Linux", *Proc. 2005 IEEE/RJS International Conference on Intelligent Robots and Systems, Edmonton, Canada, August*.

Caccavale, F., Natale, C., Siciliano, B. and Villani, L. (1998), "Resolved-acceleration control of robot manipulators: a critical review with experiments", *Robotica*, Vol. 16, pp. 565-73.

Caccavale, F., Natale, C., Siciliano, B. and Villani, L. (2001), "Achieving a cooperative behaviour in a dual-arm robot system via a modular control structure", *Journal of Robotic Systems*, Vol. 18, pp. 691-700.

Chiaverini, S., Siciliano, B. and Villani, L. (1999), "A survey of robot interaction control schemes with experimental comparison", *IEEE/ASME Transactions on Mechatronics*, Vol. 4, pp. 273-85.

Dogliani, F., Magnani, G. and Sciavicco, L. (1993), "An open architecture industrial controller", *Newsl. of IEEE Robotics and Automation Soc.*, Vol. 7 No. 3, pp. 19-21.

Grimaldi, N. (2004), "RePLiCS ver. 2.0", PRISMA Lab Internal Report.

Lippiello, V. and Villani, L. (2003), "Managing redundant visual measurements for accurate pose tracking", *Robotica*, Vol. 21, pp. 511-9.

Lippiello, V., Siciliano, B. and Villani, L. (2005), "An experimental setup for visual servoing applications on an industrial robotic cell", *Proc. 2005 IEEE/ASME International Conference on Advanced Intelligent Mechatronics, Monterey, CA, July.*

Macchelli, A. and Melchiorri, C. (2002), "Real time control system for industrial robots and control applications based on real time Linux", *Proc. 15th IFAC World Congress, Barcelona, Spain, July 2002*.

Pires, J.N., Ramming, J., Rauch, S. and Aráujo, R. (2002), "Force/torque sensing applied to industrial robotic deburring", *Sensor Review Journal*, Vol. 22 No. 3, pp. 232-41.

Zhang, H., Zhongxue, G., Brogårdh, T., Wang, J. and Isaksson, M. (2004), "Robotics technology in automotive powertrain assembly", *ABB Review*, No. 1.

Corresponding author

Luigi Villani can be contacted at: lvillani@unina.it